
Procedural Programming

Procedural Programming and Design Aspects

Ashish Kr. Layek (CST, IEST)

What Makes a Language Imperative?

- ❑ Programs written in imperative programming languages consist of
 - A program state
 - Instructions that change the program state
 - ❑ Program instructions are “imperative” in the grammatical sense of imperative verbs that express a command
-

Procedural Abstraction

- ❑ Nicholas Wirth described [imperative] programs as being “algorithms plus data structures”.
 - ❑ Algorithms become programs through the process of procedural abstraction and stepwise refinement.
 - ❑ Libraries of reusable functions support the process (functions = procedures)
 - ❑ Imperative programming + procedures = procedural programming.
-

Procedural Abstraction

- ❑ Procedural abstraction allows the programmer to be concerned mainly with the interface between the function (procedure) and what it computes, ignoring the details of how the computation is accomplished.
 - ❑ Abstraction allows us to think about *what is being done, not how* it is implemented.
-

Stepwise Refinement

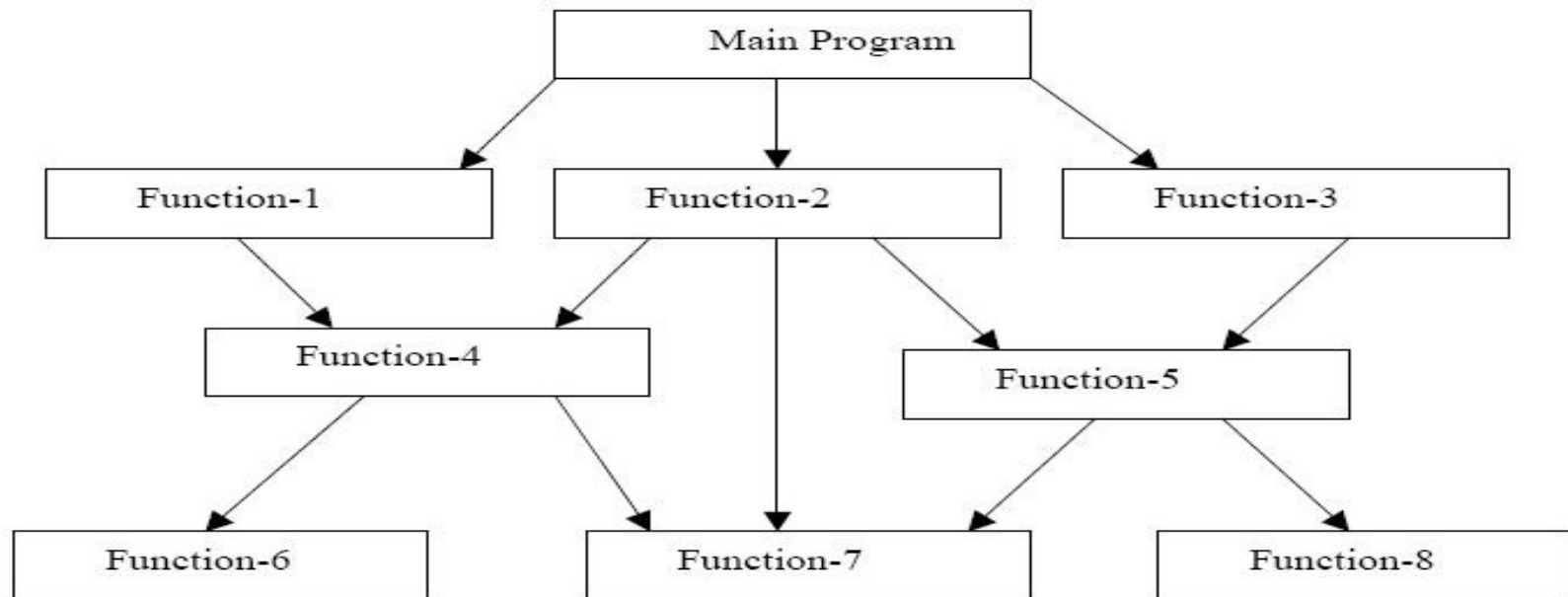
- ❑ Stepwise refinement (also called functional decomposition) uses procedural abstraction by developing an algorithm from its most general form [the abstraction] into a specific implementation.
 - ❑ Programmers start with a description of what the program should do, including I/O, and repeatedly break the problem into smaller parts, until the sub-problems can be expressed in terms of the primitive states and data types in the language.
-

Structured Programming

- ❑ A disciplined approach to imperative program design.
 - ❑ Uses procedural abstraction and top-down design to identify program components (also called modules or structures)
 - ❑ Program structures combined in a limited number of ways, so understanding how each structure works means you understand how the program works
 - ❑ Program control flow is based on decisions, sequences, loops, but...
 - ❑ Does not use goto statements
 - ❑ Modules are developed, tested separately and then integrated into the whole program.
-

Procedural Programming

- ❑ The problem is viewed as the sequence of things to be done such as reading, calculating and printing
- ❑ The primary focus is on functions - the technique of hierarchical decomposition has been used to specify the tasks to be completed for solving a problem



Procedural Programming

Contd...

- ❑ In a multi-function program, many important data items are placed as global so that they may be accessed by all the functions.
 - ❑ Global data are more vulnerable to an unintentional change by a function.
 - ❑ In a large program it is very difficult to identify what data is used by which function.
 - ❑ In case we need to revise an external data structure, we also need to revise all functions that access the data. This provides an opportunity for bugs to creep in.
 - ❑ Can not model real world problems very well - functions are action-oriented and do not really corresponding to the element of the problem
-

Procedural Programming Characteristics

- ❑ Emphasis is on doing things (algorithms).
 - ❑ Large programs are divided into smaller programs known as functions.
 - ❑ Most of the functions share global data.
 - ❑ Data move openly around the system from function to function.
 - Data doesn't have a owner – many function can modify data
 - ❑ Functions transform data from one form to another.
 - Difficulty in maintaining data integrity
 - Difficult to pinpoint bug sources when data is corrupted
 - ❑ Employs top-down approach in program design.
-

Requirement Phase: Example LMS

❑ Development of a small-scale Library Management System(LMS).

❑ Supported Functionalities:

1. Add user (membership)
2. Modify User
3. Delete User
4. Search Book
5. Add Books (record)
6. Modify Book
7. Delete Book.
8. Issue Book (to a user)
9. Return Book
10. Renew Book.

Functional Requirements
(What the system should do)

Note:

✓ Needs detailing of each requirement

Requirement Phase: Example LMS

❑ Development of a small-scale Library Management System(LMS).

❑ Non-functional aspects

1. Performance Requirements
2. Scalability Requirements
3. Availability & Reliability
4. Security Requirements
5. Usability Requirements
6. Maintainability & Extensibility
7. Portability
8. Data Integrity & Consistency
9. Logging & Auditing
10. Backup & Recovery

Non-functional Requirements
(How a system should work)

Note:

- ✓ Needs detailing of each requirement
- ✓ All NFRs are not mandatory

Design phase

- ❑ Goal: transform SRS (Requirement) document into a form easily implementable using some programming language

 - ❑ Items developed during this phase
 - Different module required to implement the design solution
 - Module structure
 - Control relationships among modules
 - Interface among modules
 - Data structures and algorithms of individual modules
-

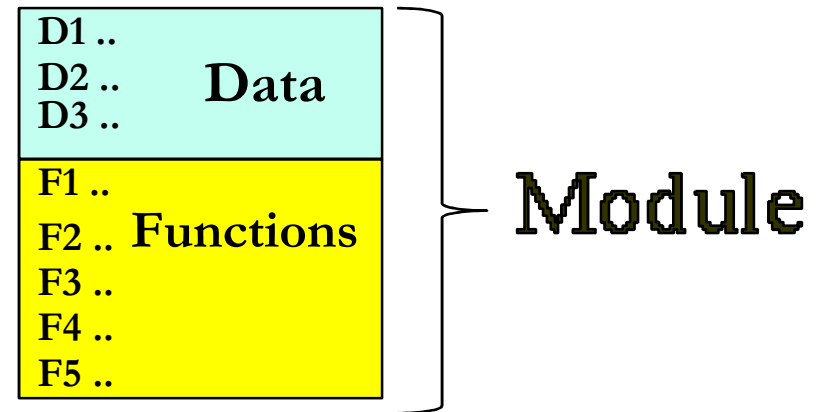
Two stages of design

- ❑ Preliminary or high-level design
 - Outcome is program structure (called software architecture)
 - Several notations available to represent program structure:
Structure chart, Jackson diagram, Warnier-Orr diagram

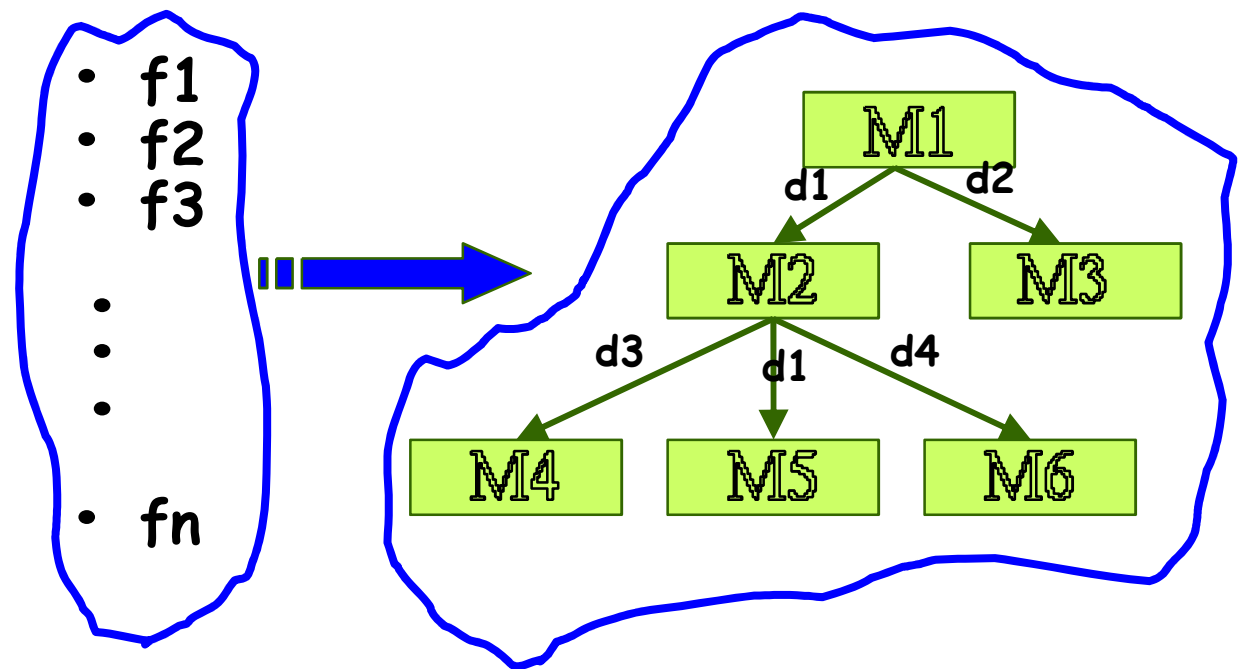
 - ❑ Detailed design or low-level design
 - Data structure & algorithm of each module
 - Outcome is called module specification
-

Module

- ❑ A module consists of:
 - Several functions
 - Associated data structures.



- ❑ High-level design maps functions into modules $\{f_i\}$ $\{M_j\}$



Good software design

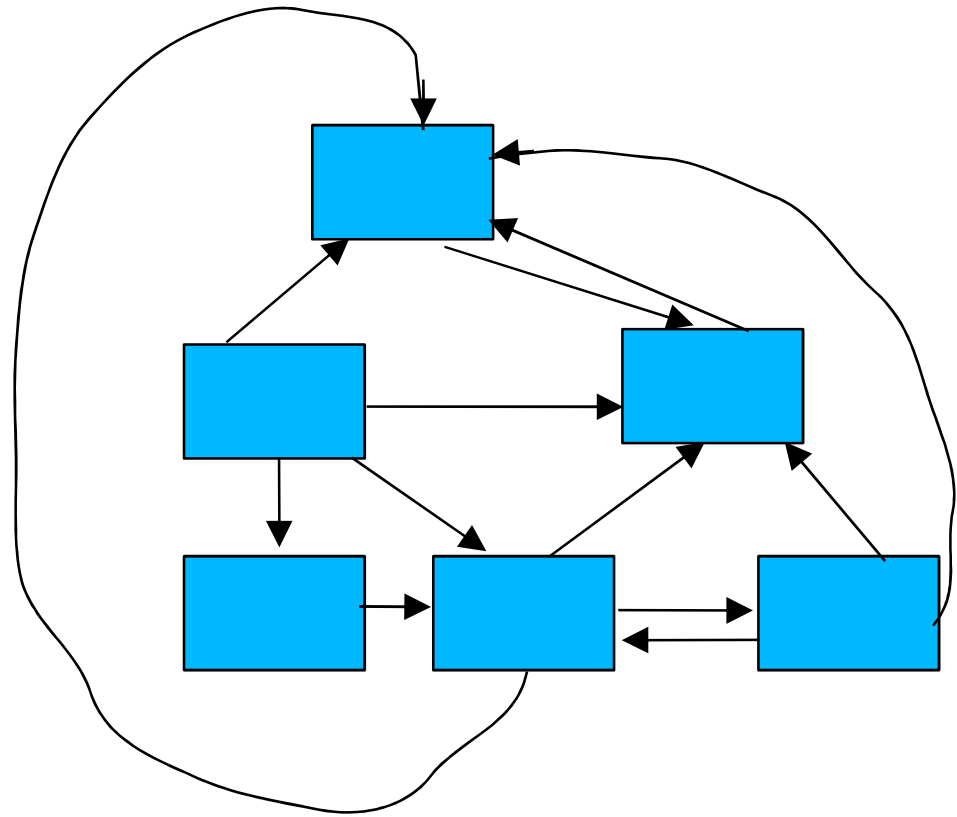
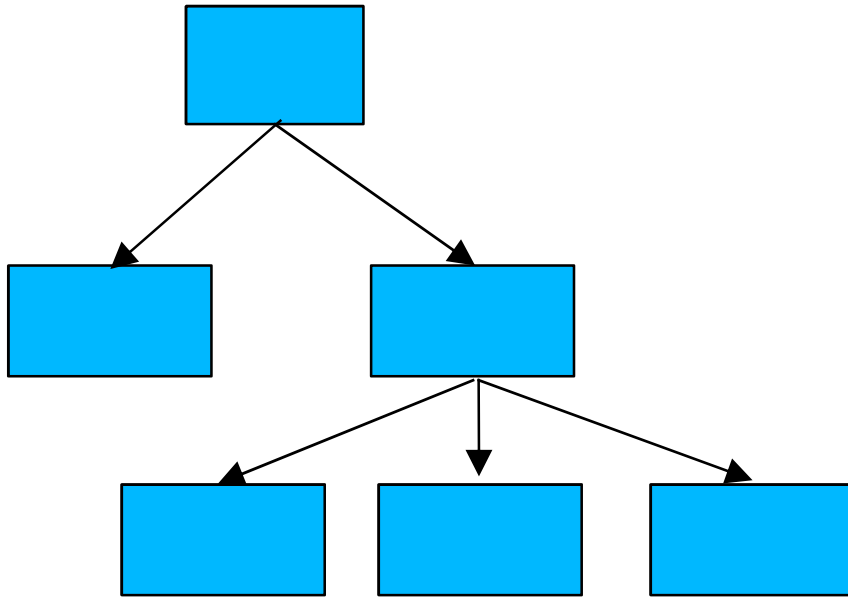
- ❑ No unique design for a system
 - Different designs possible for same system even in same design methodology is used
 - ❑ Characteristics of good design
 - Implement all required functionalities correctly
 - Efficient
 - Easily understandable
 - ❖ Use consistent and meaningful names for various design components
 - ❖ Modularity
 - Easily amenable to change / maintainable
-

Modularity

- ❑ Design should consist of a cleanly decomposed set of modules
 - Modules should be almost independent of each other
 - Modules should be neatly arranged in a hierarchy

 - ❑ In technical terms, modules should display
 - High cohesion
 - Low coupling
 - Low fan-out
 - Abstraction
-

Good and bad decomposition into modules



Cohesion & Coupling

❑ Cohesion

- Measure of **functional strength** of a module
- A cohesive module performs a single / few related tasks

❑ Coupling between two modules

- Measure of the **degree of interdependence** or interaction between the two modules. How closely two modules interact or how interdependent they are
 - ❖ Depends on the interface between the modules
 - ❖ No objective metrics, but a subjective classification
- A module having low coupling is **functionally independent** of other modules

❑ No ways to quantitatively measure cohesion & coupling, only subjective classification

Classification of cohesiveness

Functional
Sequential
Communicational
Procedural
Temporal
Logical
Coincidental



Degree of cohesion

Cohesion

Cont...

- ❑ **Coincidental Cohesion:** The module performs a set of tasks:
 - Which relate to each other very loosely, if at all.
 - ✓ The module contains a random collection of functions.
 - ✓ Functions have been put in the module out of pure coincidence without any thought or design.

 - ❑ **Logical Cohesion:** All elements of the module perform similar operations:
 - e.g. error handling, data input, data output, etc.
 - A precise example of logical cohesion:
 - ❖ A set of print functions to generate an output report arranged into a single module.
-

Cohesion

Cont...

- ❑ **Temporal Cohesion:** The module contains tasks that are related by the fact:
 - All the tasks must be executed in the same time span.
 - Example: The set of functions responsible for
 - ❖ Initialization,
 - ❖ start-up, shut-down of some process, etc.

 - ❑ **Procedural Cohesion:** The set of functions of the module:
 - All part of a procedure (algorithm)
 - Certain sequence of steps have to be carried out in a certain order for achieving an objective,
 - ❖ e.g. the algorithm for encoding/decoding a message.
-

Cohesion

Cont...

- ❑ **Communicational Cohesion:** All functions of the module:
 - Reference or update the same data structure,
 - Example: The set of functions defined on an array or a stack.

 - ❑ **Sequential Cohesion:** Elements of a module form different parts of a sequence
 - Output from one element of the sequence is input to the next.
 - Example: CreateOrder() → checkItemAvailability() → PlaceOrderOnVendor()
-

Cohesion

Cont...

- ❑ **Functional Cohesion:** Different elements of a module cooperate:
 - To achieve a single function,
 - e.g. managing an employee's pay-roll.
 - ❖ `computeOvertime()`
 - ❖ `computeWorkhours()`
 - ❖ `compute deduction()`

 - ❑ **When a module displays functional cohesion,**
 - We can describe the function using a single sentence.
-

Classification of Coupling

Data
Stamp
Control
Common
Content



Degree of coupling

Coupling

Cont...

- ❑ **Data Coupling:** Two modules are data coupled,
 - If they communicate via a parameter:
 - ❖ an elementary data item, e.g an integer, a float, a character, etc.
 - The data item should be problem related:
 - ❖ Not used for control purpose.

 - ❑ **Stamp Coupling :** Two modules communicate via a composite data item
 - such as a record in PASCAL or a structure in C.
-

Coupling

Cont...

- ❑ **Control Coupling:** Data from one module is used to direct the Order of instruction execution in another.
 - Example: A control information (flag) is passed from one module to another. A flag set in one module and tested in another module.

 - ❑ **Common Coupling:** Two modules are share some global data.

 - ❑ **Content Coupling :** If two module share code,
 - e.g, branching from one module into another module.
-

Design Phase: Example LMS

❑ Development of a small-scale Library Management System(LMS).

❑ Supported Functionalities and module breakdown

1. Add User (member)

2. Modify User

3. Delete User

4. Search Book

5. Add Books (record)

6. Modify Book

7. Delete Book.

8. Issue Book (to a user)

9. Return Book

10. Renew Book.

User Management Module

Book Management Module

Circulation Module

Design Phase: Example LMS

❑ Development of a small-scale Library Management System(LMS).

1. Performance Requirements
2. Scalability Requirements
3. Availability & Reliability
4. Security Requirements
5. Usability Requirements
6. Maintainability & Extensibility
7. Portability
8. Data Integrity & Consistency
9. Logging & Auditing
10. Backup & Recovery

Policy / Rules Module

Data Access Module

Presentation / Interface
Module

Logging Module

and some more ...

Functional Independence

- ❑ A module having high cohesion and low coupling is said to be functionally independent of other modules.
- ❑ It means the functionally independent module performs
 - A single task or function.
 - It has minimal interaction with other modules.

Advantages of functional independence

- ❑ Better understandability of design
 - Different modules can be understood in isolation
 - ❑ Reduces error propagation
 - ❑ Reuse of modules possible
-

Hierarchy of modules

- ❑ Control hierarchy

- Organization and invocation relationships among modules
- Also called program structure / software architecture

- ❑ Most common representation: a tree-like diagram called structure chart

- ❑ Properties of a neat hierarchy of modules

- Low fan-out
 - Abstraction
-

Characteristics of module structure

- ❑ Layering: modules arranged in layers
 - Module M calls module N, $\rightarrow$ M is in layer above N
 - M is super-ordinate to N, N is subordinate to M

 - ❑ Control abstraction: layered design
 - A module should invoke functions of modules only in the layer immediately below itself
 - Modules at lower layers should not invoke the modules above it
-

Characteristics of module structure

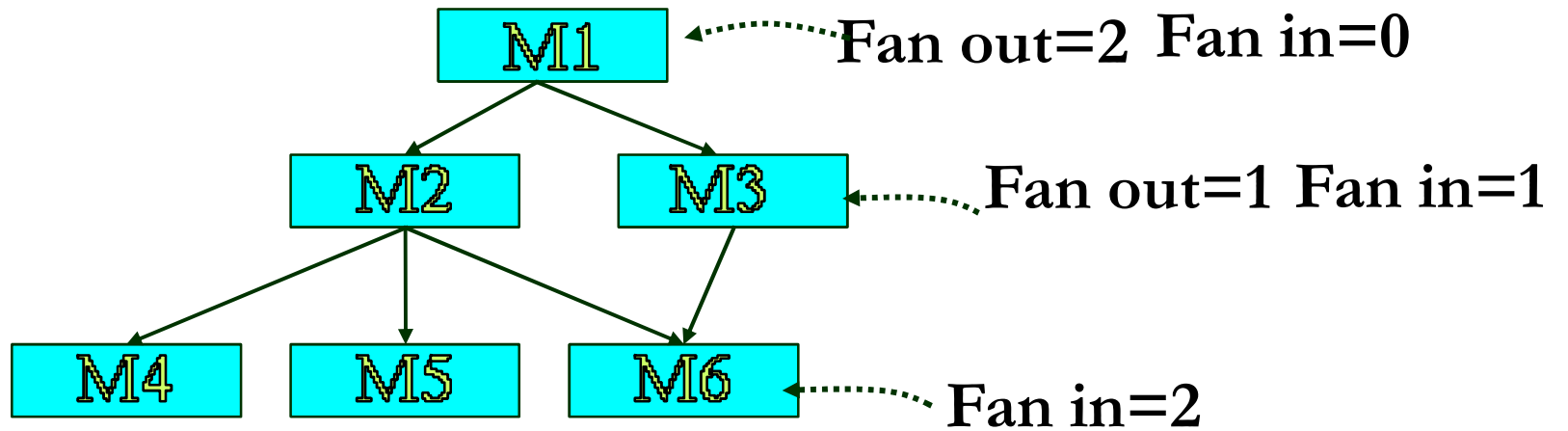
❑ Fan-out

- A measure of the number of modules directly controlled by given module
- In general, modules with high fan-out likely to lack cohesion, hence discouraged

❑ Fan-in

- Indicates how many modules directly invoke a given module.
 - High fan-in represents code reuse and is in general encouraged
-

Module Structure



Design approaches

- ❑ High level design: mapping functional requirements (in SRS) into modules, such that
 - Each module has high cohesion
 - Coupling among modules is as low as possible
 - Modules are organized in a neat hierarchy

 - ❑ Two fundamentally different software design approaches
 - Function-oriented design
 - Object-oriented design
-

Function-oriented design

- ❑ A system looked upon as something that performs a set of functions
 - Each function successively refined into more detailed functions
 - Functions are mapped to a module structure

 - ❑ System state is centralized, accessible to different functions
-

Object-oriented design

- ❑ System is viewed as a collection of classes & their objects (e.g. real-world entities)
 - Objects have their own internal data
 - Objects communicate by message passing
- ❑ System state is decentralized among the objects

The two design approaches

❑ Gady Booch:

- “Identify verbs if you are after procedural design and nouns if you are after object-oriented design.”

❑ The two approaches are complementary rather than competing

- Both techniques applicable at different stages in the design process
 - Even in an object-oriented design, there is a hierarchy of function inside each class
-